মেট্রোপলিটন ইউনিভার্সিটি
Metropolitan University

# Department of Computer Science and Engineering

| Course Code: CSE222 | Credits: 1.5 |
|---|---|
| Course Name: Object Oriented Programming Lab | Faculty: FRS |

## Lab 04 – Inheritance and Abstract

**Task 1:** Create a class called "Song" with final attributes "title" and "artist" that cannot be modified after initialization. Add a static counter "playCount" that tracks how many songs have been played across the application. Implement a method "play()" that increments this counter whenever any song is played. Create a simple program that instantiates several songs and plays them, then displays the total play count.

*Sample Output:*

*Playing "Bohemian Rhapsody" by Queen...*

*Playing "Shape of You" by Ed Sheeran...*

*Playing "Bohemian Rhapsody" by Queen...*

*Total songs played: 3*

**Task 2:** Create a class hierarchy for a library system. Start with a parent class "LibraryItem" containing a final attribute "itemId" and static attribute "totalItems" that keeps track of all items in the library. Create a child class "Book" that inherits from LibraryItem and adds attributes for "title" and "author". Implement a static method "getTotalItemCount()" in the LibraryItem class that returns the current count. Create a program that creates several books and displays the total item count.

*Sample Output:*

*Added book: "1984" by George Orwell.*

*Added book: "The Catcher in the Rye" by J.D. Salinger.*

*Added book: "To Kill a Mockingbird" by Harper Lee.*

*Total library items: 3*

**Task 3:** Create an abstract class called "Shape" with a final attribute "color" and static attribute "shapeCount". Include abstract methods "calculateArea()" and "calculatePerimeter()". Create two subclasses "Circle" and "Rectangle" that inherit from Shape and implement the abstract methods. Add a static method "getTotalShapes()" to the Shape class. Create a program that instantiates various shapes, calculates their areas and perimeters, and displays the total count of shapes created.

*Sample Output:*

*Circle (Red): Area = 78.54, Perimeter = 31.42*

*Rectangle (Blue): Area = 20.0, Perimeter = 18.0*

*Total shapes created: 2*

**Task 4:** Create a multi-level inheritance hierarchy for vehicles. Start with an abstract base class "Vehicle" with final attributes "manufacturerName" and "modelYear". Add a static counter "totalVehicles". Create an intermediate abstract class "PoweredVehicle" that extends Vehicle and adds an abstract method "startEngine()". Finally, create concrete classes "Car" and "Motorcycle" that extend PoweredVehicle and implement the abstract methods. Each concrete class should increment the static counter in its constructor. Create a static method in the Vehicle class to report statistics about all vehicles created.

*Sample Output:*

*Car - Toyota Corolla (2022): Engine started.*

*Motorcycle - Yamaha R1 (2021): Engine started.*

*Total vehicles created: 2*

**Task 5:** Create an abstract class called "Movie" with final attributes "title" and "director" that cannot be modified after initialization, a static attribute "totalMovies" that keeps track of the number of movie instances created across the application, a static method "getMovieCount()" that returns the total count of all movie objects, and abstract methods "calculateRating()" and "displayInfo()"; then create two subclasses, "ActionMovie" and "ComedyMovie," that inherit from the Movie class, implementing the abstract methods for each subclass, where ActionMovie includes a final attribute "stuntsCoordinator" and a static attribute "dangerLevel", while ComedyMovie includes a final attribute "humorStyle" and a static method "getLaughterRating()", ensuring that both subclasses correctly increment the totalMovies count when new instances are created.

*Sample Output:*

*Action Movie: "Mad Max: Fury Road" directed by George Miller.*

*Stunt Coordinator: Tom Hardy*

*Danger Level: 9/10*

*Calculated Rating: 8.5/10*

*Comedy Movie: "Superbad" directed by Greg Mottola.*

*Humor Style: Teen Comedy*

*Laughter Rating: 7.8/10*

*Calculated Rating: 7.5/10*

*Total movies created: 2*

**Task 6:** Create an abstract class called "Book" with final attributes "title" and "author" that remain constant throughout the object's lifecycle, a static attribute "totalBooks" that maintains a count of all book instances, a static method "getTotalPages()" that calculates and returns the sum of pages across all books, and abstract methods "calculateReadingTime()" and "generateSummary()"; then create two subclasses, "FictionBook" and "NonFictionBook," that inherit from the Book class, implementing the abstract methods for each subclass, where FictionBook includes a final attribute "genre" and a static method "getMostPopularGenre()", while

NonFictionBook includes a final attribute "subject" and a static collection of "academicReferences", with the reading time calculation differing based on the book type, with fiction generally requiring less time per page than non-fiction.

*Sample Output:*

*Fiction Book: "The Hobbit" by J.R.R. Tolkien.*

*Genre: Fantasy*

*Estimated reading time: 5 hours*

*Summary: A hobbit embarks on an unexpected journey.*

*Non-Fiction Book: "Sapiens" by Yuval Noah Harari.*

*Subject: Human History*

*Estimated reading time: 8 hours*

*Academic References: [Evolution, Anthropology, Sociology]*

*Summary: A deep dive into the history of humankind.*

*Total books: 2*

*Total pages across all books: 850*

*Most popular genre: Fantasy*